AGILE DEVELOPMENT PROCESS AND DEVOPS

LAB ASSIGNMENT – 02

NAME : SIVASHREE.PL **CODE : ISWE406P**

REG NO : 23MIS0117 **SLOT : C1[L49+L50]**

JENKINS

1. Learning Goal

Understand **what Jenkins is** and run a job successfully.

Stepwise Tasks

1. What is CI/CD? (Real-world example)

CI/CD (Continuous Integration / Continuous Delivery or Deployment) is a DevOps practice in which **code changes are automatically built, tested, and deployed** so that software updates reach users **quickly, safely, and frequently**.

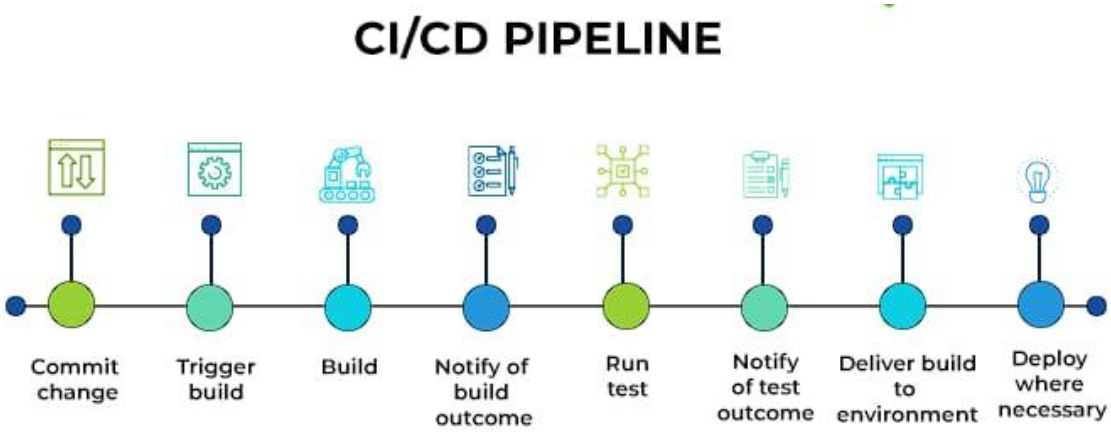
Real-World Example: Food Delivery App (Swiggy/Zomato)

Scenario

A developer fixes a bug in the **online payment feature** of a food delivery app.

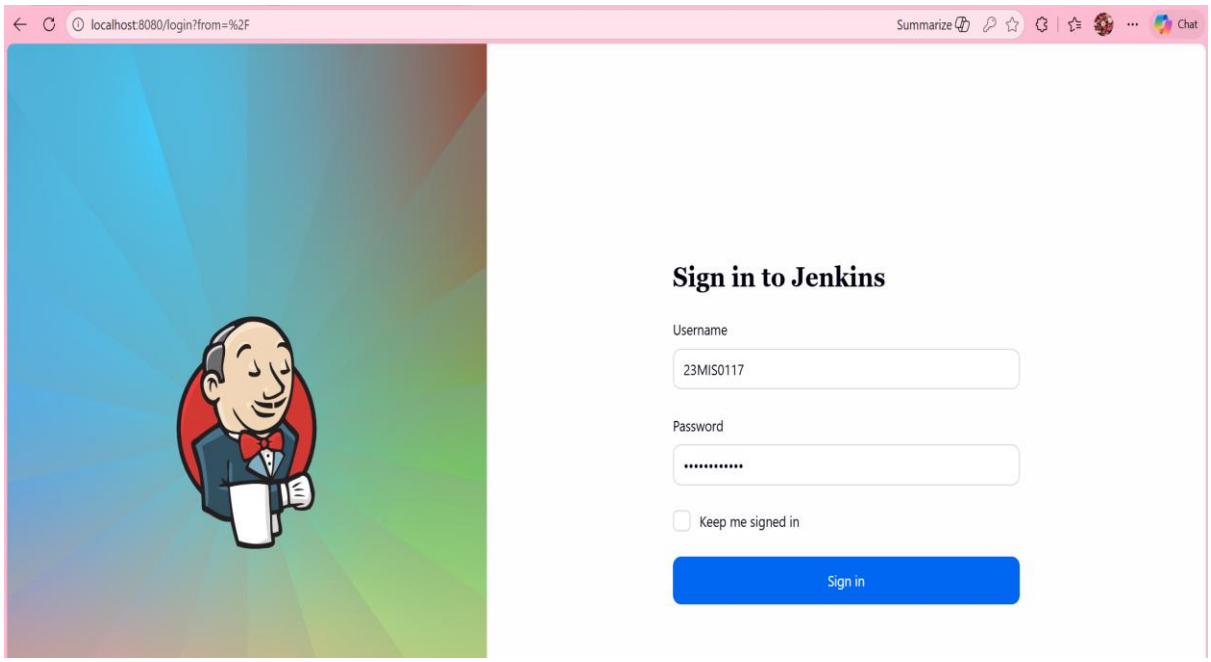
How CI/CD Works Here

1. The developer updates the payment code and **pushes it to GitHub**.
2. **Continuous Integration (CI)** automatically:
   - Builds the application
   - Runs automated tests to ensure payments work correctly
3. If all tests pass, **Continuous Deployment (CD)** automatically:
   - Deploys the updated app to the live server
4. Users immediately get a **bug-free payment experience** without app downtime.



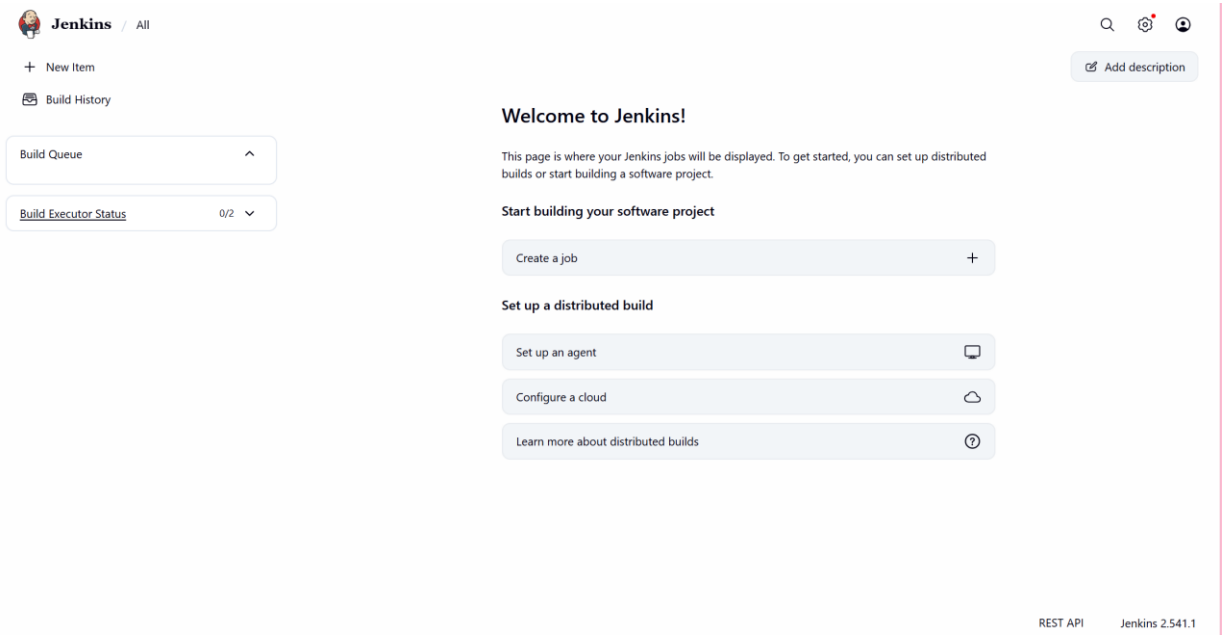
2. Install Jenkins (Windows / Linux)

```
C:\Users\sivas>java -version
java version "25.0.2" 2026-01-20 LTS
Java(TM) SE Runtime Environment (build 25.0.2+10-LTS-69)
Java HotSpot(TM) 64-Bit Server VM (build 25.0.2+10-LTS-69, mixed mode, sharing)

C:\Users\sivas>sc query jenkins

SERVICE_NAME: jenkins
        TYPE               : 10  WIN32_OWN_PROCESS
        STATE              : 4  RUNNING
                                (STOPPABLE, NOT_PAUSABLE, ACCEPTS_SHUTDOWN)
        WIN32_EXIT_CODE    : 0  (0x0)
        SERVICE_EXIT_CODE  : 0  (0x0)
        CHECKPOINT         : 0x0
        WAIT_HINT          : 0x0
```

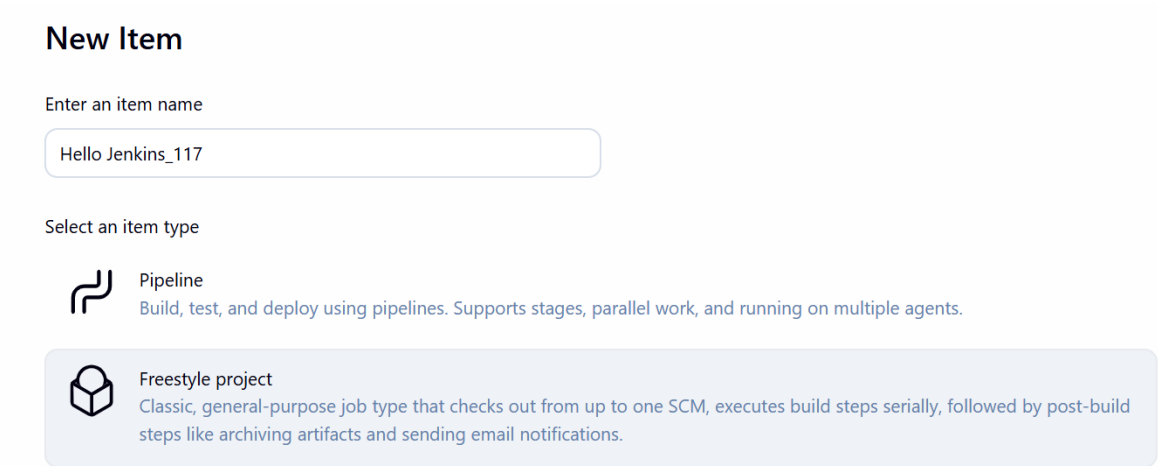


3. Jenkins UI overview

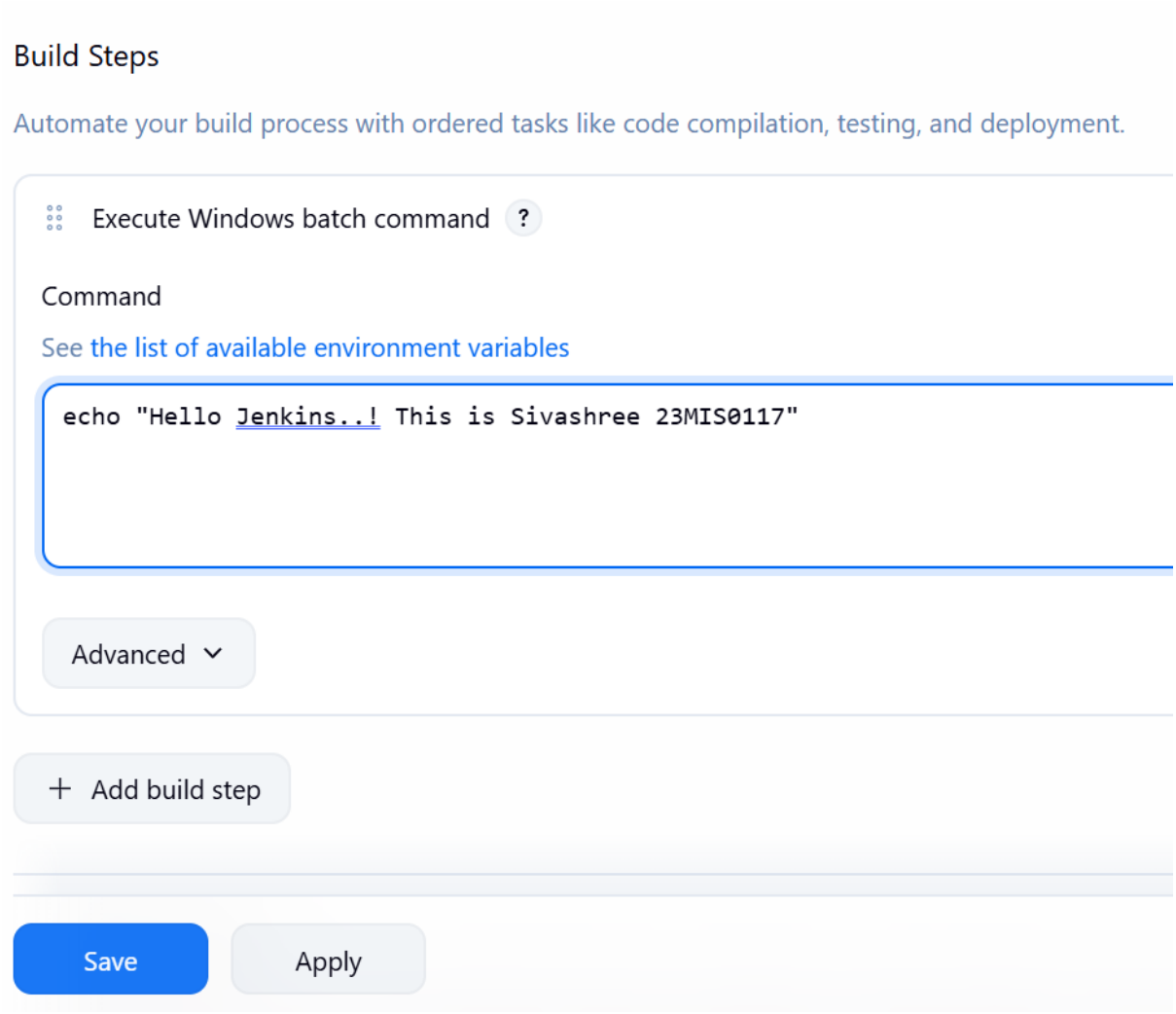
Section	Purpose
Dashboard	View all jobs
New Item	Create jobs
Build History	Previous builds
Manage Jenkins	Configuration
Console Output	Build logs



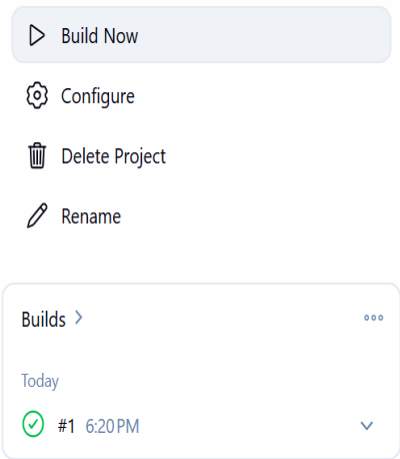
4. Create **Freestyle Job**



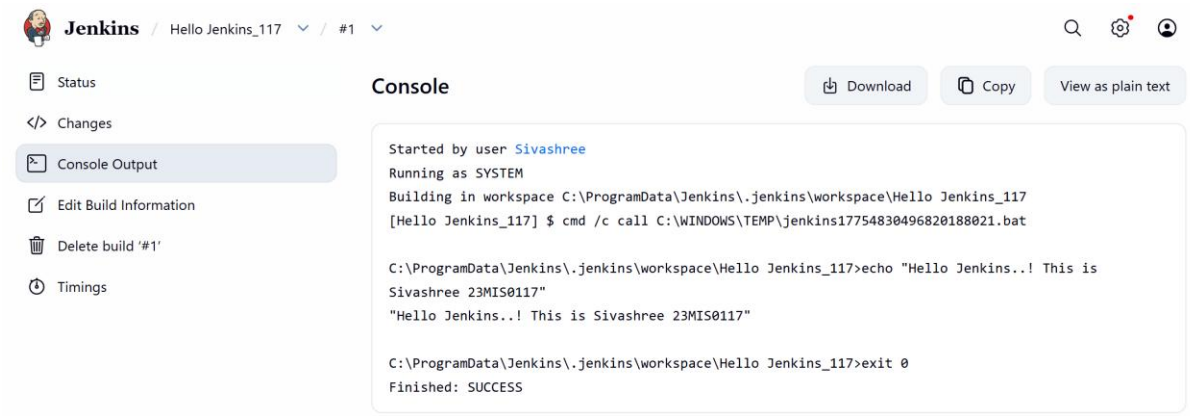
5. Add simple build step:
   - Windows: `echo Hello Jenkins`



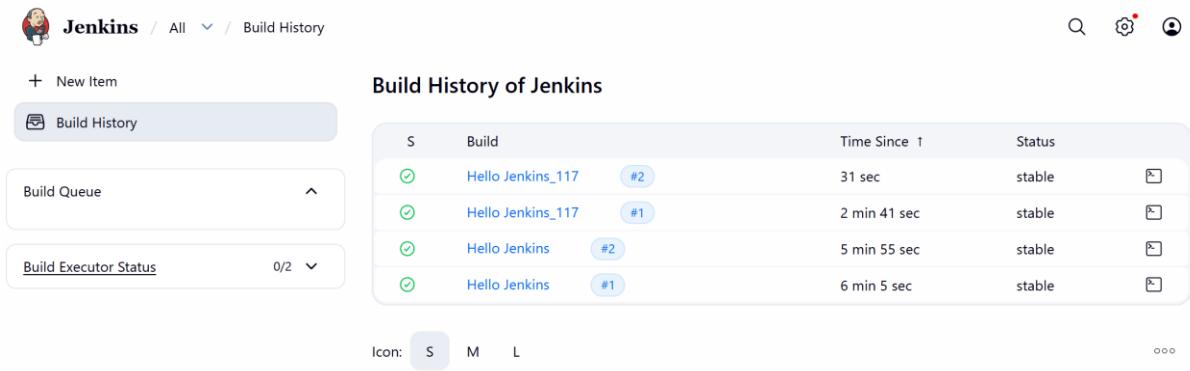
6. Run job manually



7. View:
   - Console Output



   - Build History



Outcome

A learner can install Jenkins and run a basic job.

2. Learning Goal

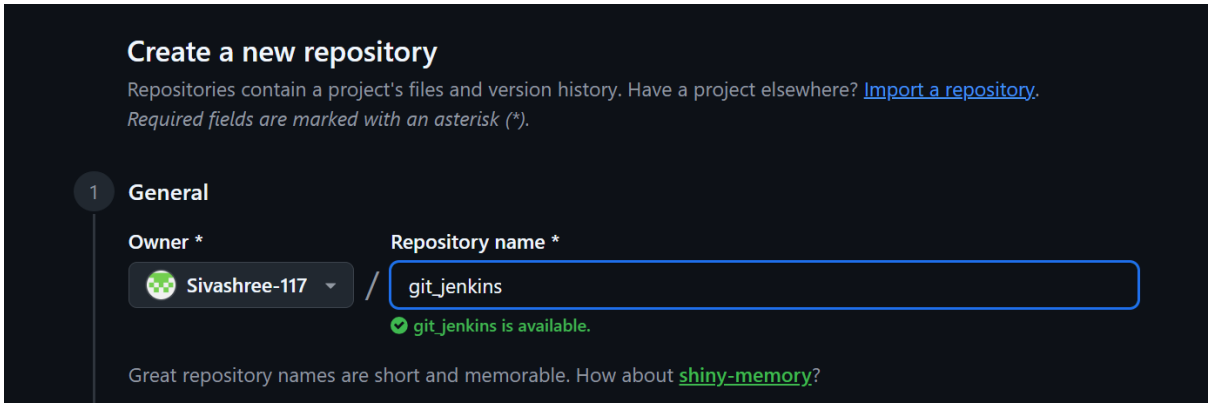
Work with **source code and build automation**

Stepwise Tasks

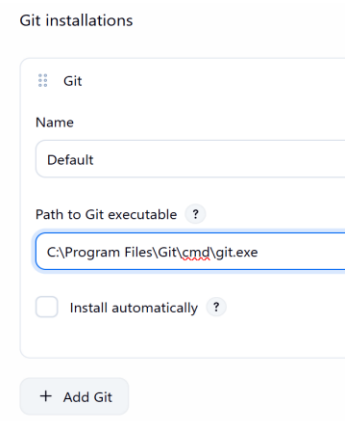
1. Install Git

```
C:\Users\sivas>git --version
git version 2.52.0.windows.1
```

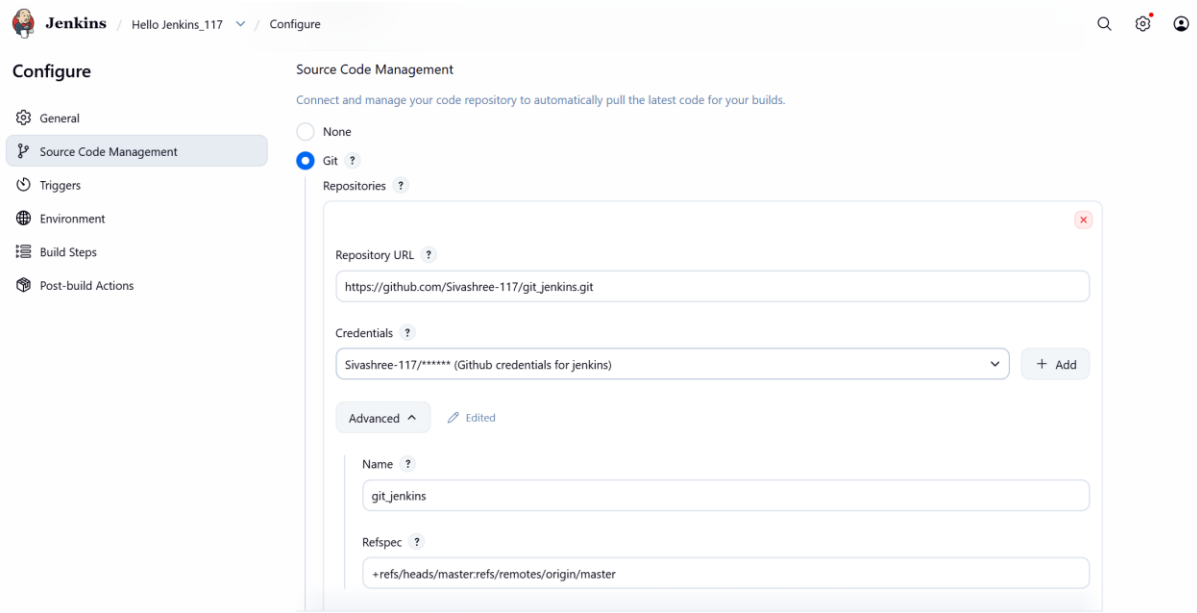
2. Create GitHub repository



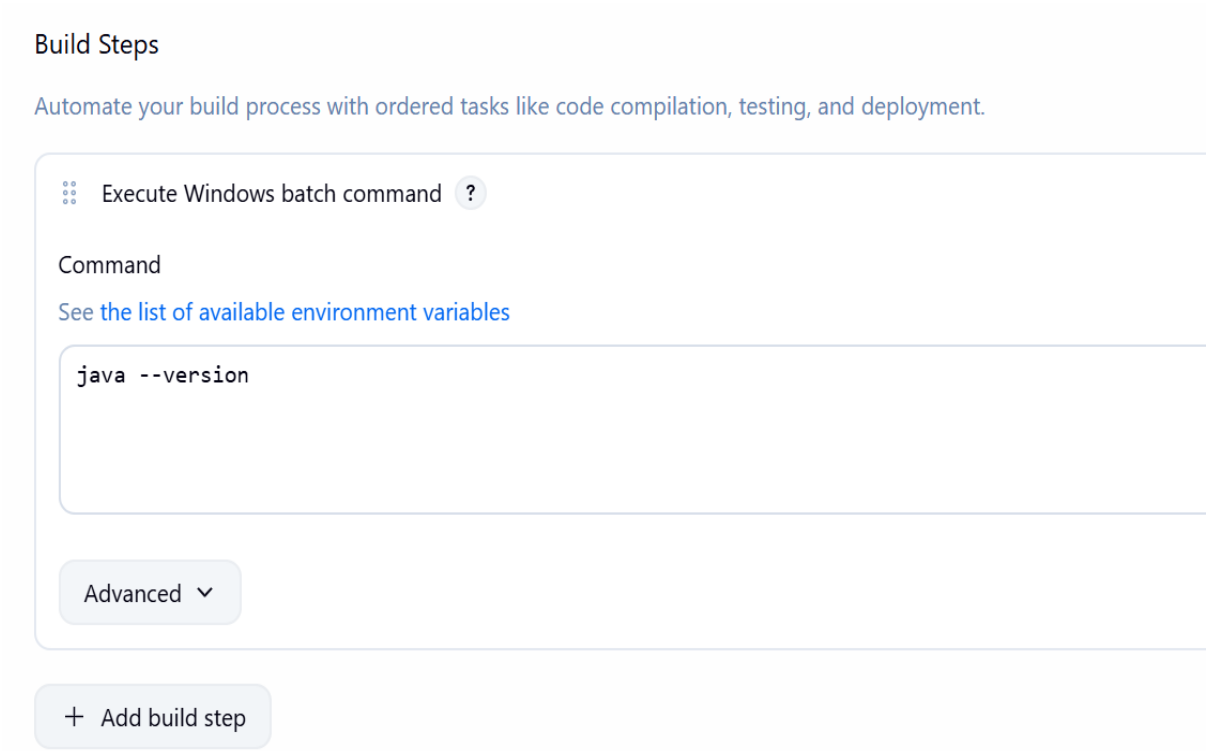
3. Configure **Git in Jenkins**



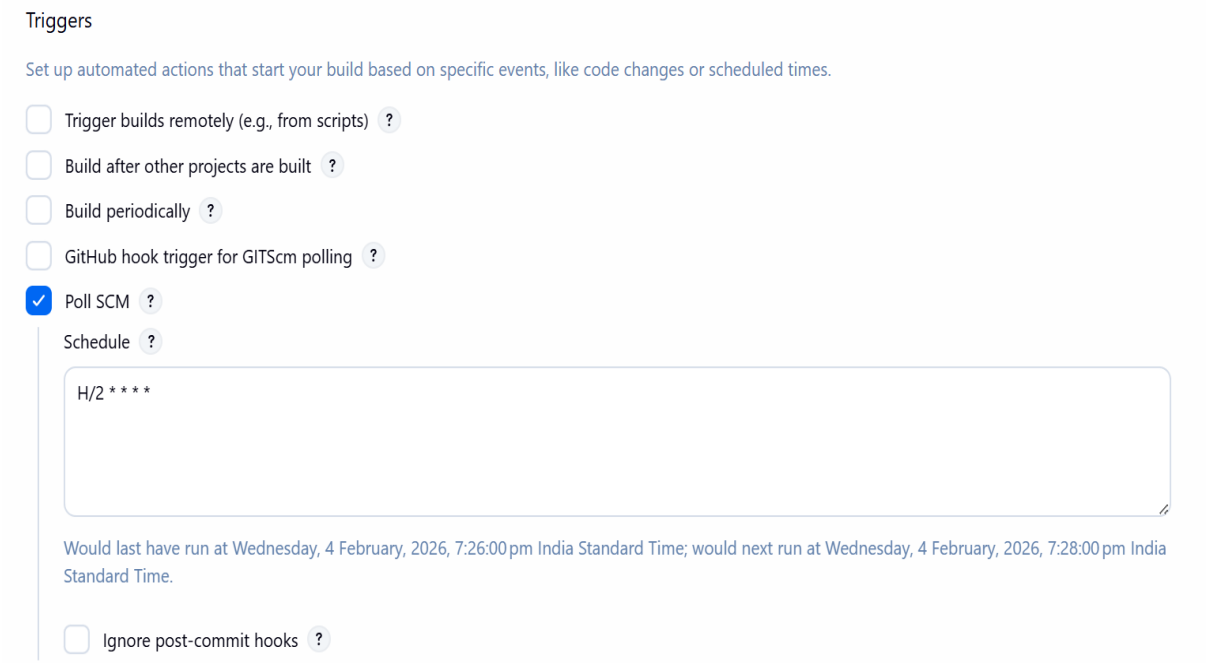
4. Create Freestyle Job:
   - Source Code Management → Git



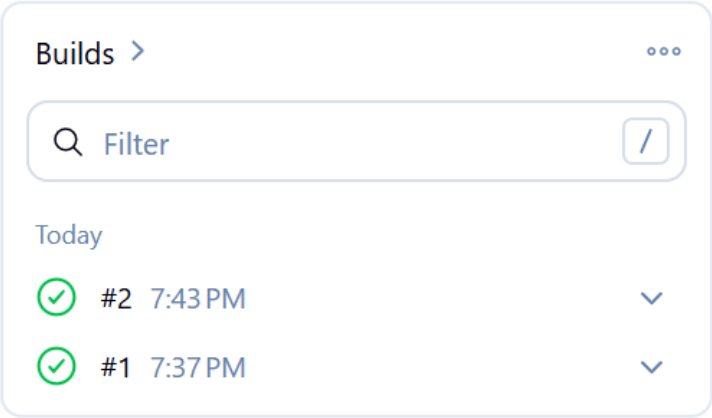
5. Add build step:
   - Windows batch

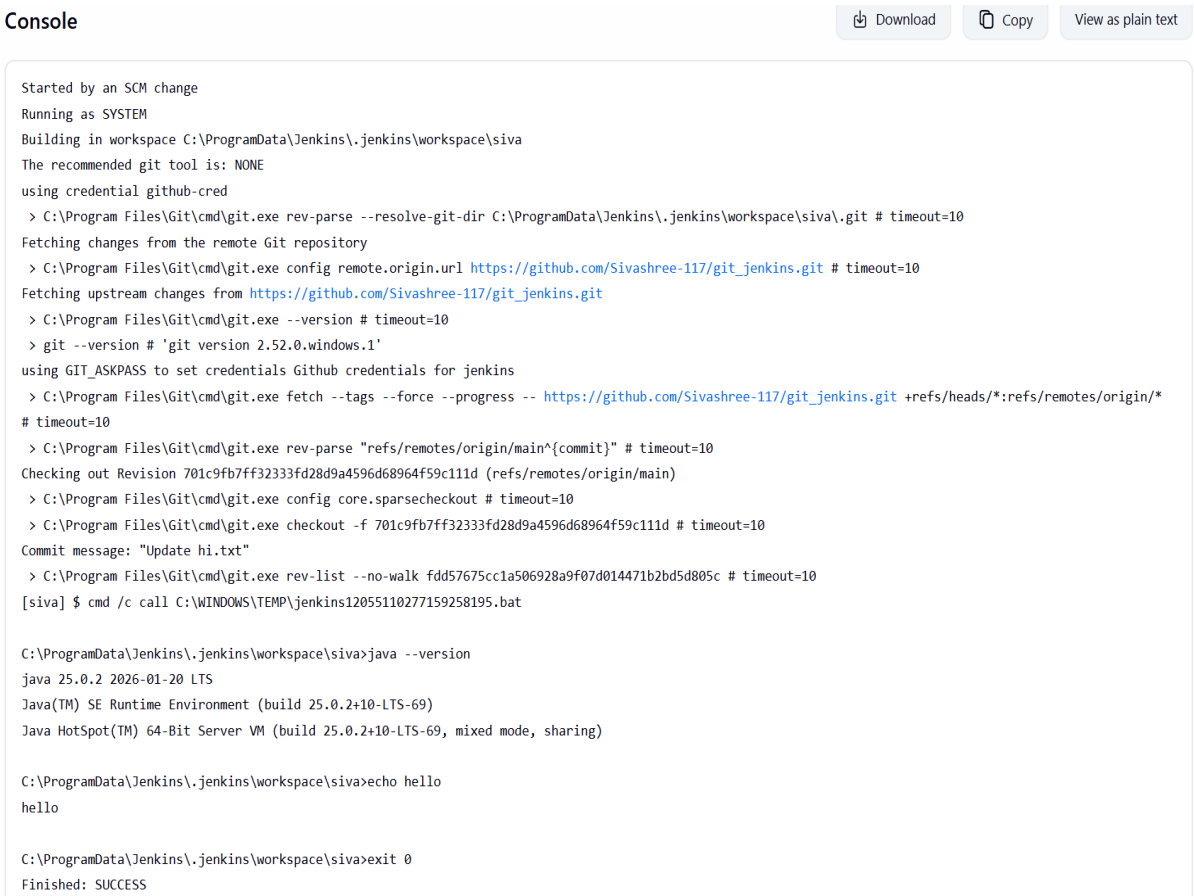


6. Poll SCM



7. Build automatically on code change





Outcome

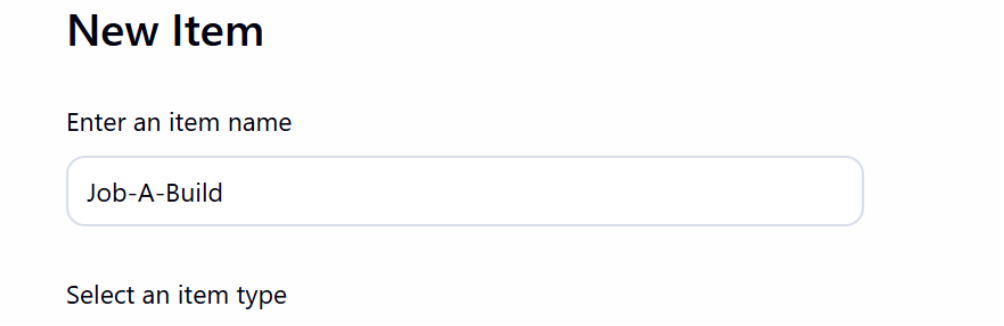
Learner connects Jenkins with GitHub.

3. Learning Goal

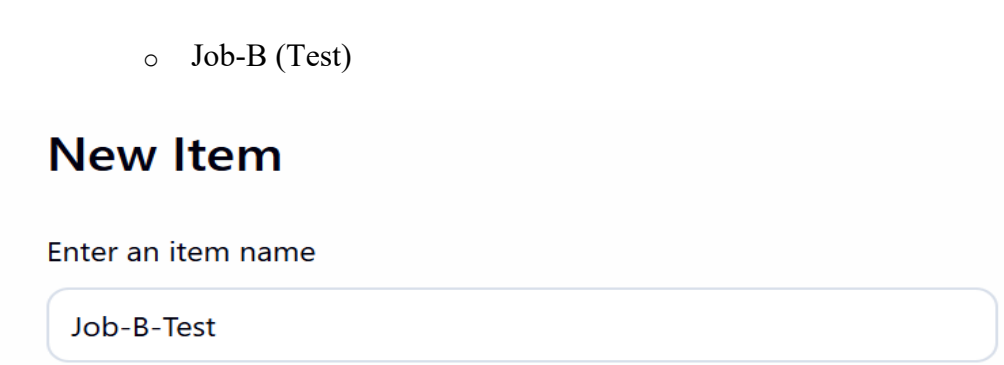
Understand **job flow visually**

Stepwise Tasks

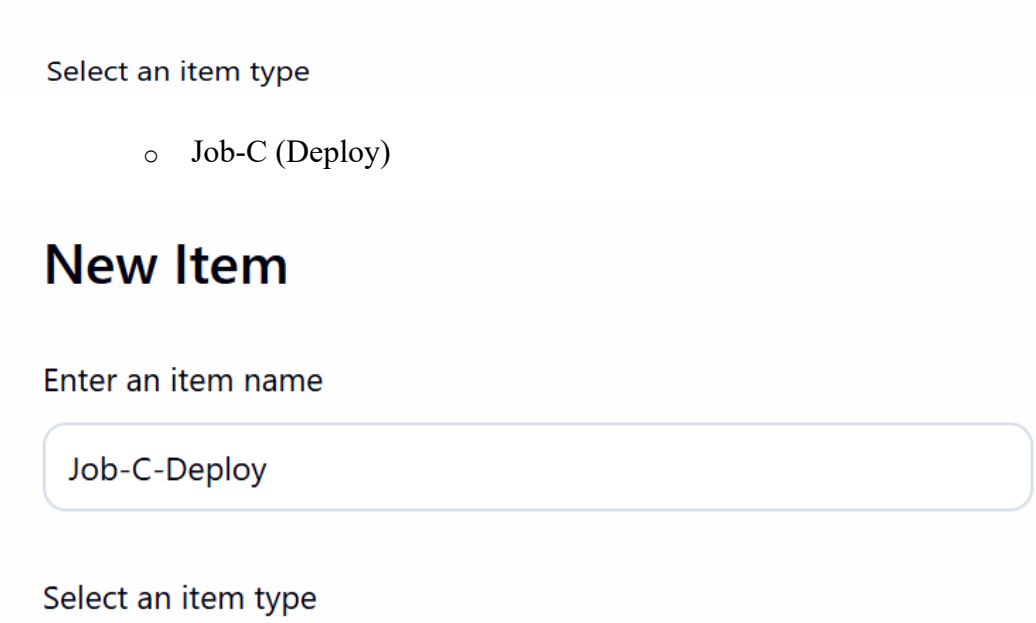
1. Create multiple Freestyle jobs:
    - Job-A (Build)



    - Job-B (Test)



    - Job-C (Deploy)



2. Use:
   - Post-build action → Trigger other projects

Execute Windows batch command

Command

See the list of available environment variables

```
echo --Build Stage--
echo Compiling source code..
```

Advanced

+ Add build step

Post-build Actions

Define what happens after a build completes, like sending notifications, archiving artifacts, or triggering other jobs.

Build other projects

Projects to build

Job-B-Test

Trigger only if build is stable

3. View job chaining visually

Console

```
Started by user Sivashree
Running as SYSTEM
Building in workspace C:\ProgramData\Jenkins\.jenkins\workspace\Job-A-Build
[Job-A-Build] $ cmd /c call C:\WINDOWS\TEMP\jenkins6303576109228977297.bat

C:\ProgramData\Jenkins\.jenkins\workspace\Job-A-Build>echo --Build Stage--
--Build Stage--

C:\ProgramData\Jenkins\.jenkins\workspace\Job-A-Build>echo Compiling source code..
Compiling source code..

C:\ProgramData\Jenkins\.jenkins\workspace\Job-A-Build>exit 0
Triggering a new build of Job-B-Test
Finished: SUCCESS
```

Console

```
Started by user Sivashree
Running as SYSTEM
Building in workspace C:\ProgramData\Jenkins\.jenkins\workspace\Job-B-Test
[Job-B-Test] $ cmd /c call C:\WINDOWS\TEMP\jenkins10128356484779730585.bat

C:\ProgramData\Jenkins\.jenkins\workspace\Job-B-Test>echo --Test Stage--
--Test Stage--

C:\ProgramData\Jenkins\.jenkins\workspace\Job-B-Test>echo Running test cases
Running test cases

C:\ProgramData\Jenkins\.jenkins\workspace\Job-B-Test>exit 0
Triggering a new build of Job-C-Deploy
Finished: SUCCESS
```

Job-A-Build

Downstream Projects

Job-B-Test

Permalinks

- Last build (#3), 1 min 29 sec ago
- Last stable build (#3), 1 min 29 sec ago
- Last successful build (#3), 1 min 29 sec ago
- Last completed build (#3), 1 min 29 sec ago

Job-B-Test

Upstream Projects

Job-A-Build

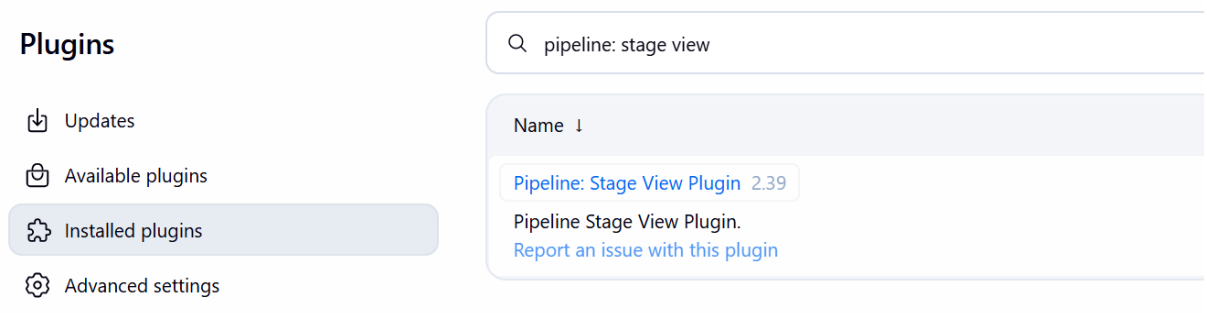
Downstream Projects

Job-C-Deploy

Permalinks

- Last build (#3), 1 min 24 sec ago
- Last stable build (#3), 1 min 24 sec ago
- Last successful build (#3), 1 min 24 sec ago
- Last completed build (#3), 1 min 24 sec ago

4. Install **Pipeline Stage View**



5. Observe stage-wise execution

Outcome

Learner understands pipeline flow visually.

4. Learning Goal

Use **Jenkins Pipeline & Jenkinsfile**

Stepwise Tasks

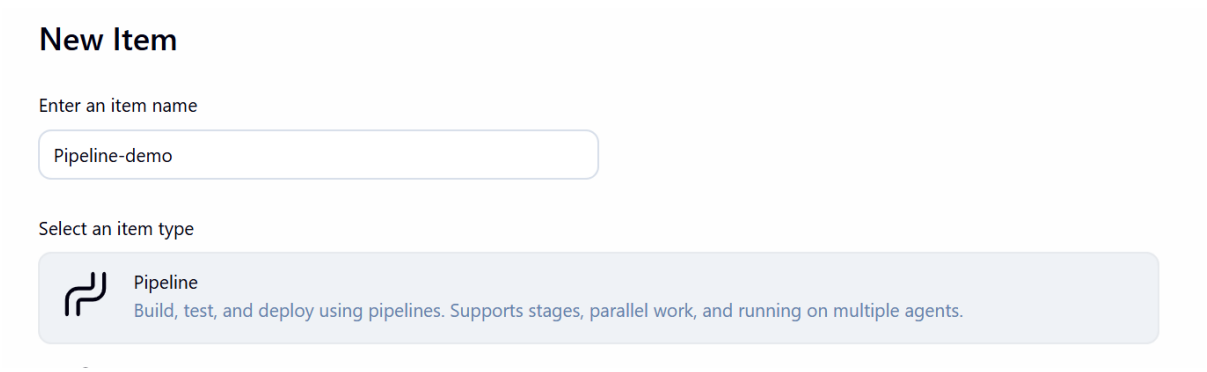
1. What is Jenkins Pipeline?

Jenkins Pipeline is a suite of automated steps written as code to define the CI/CD process.
It automates building, testing, and deploying applications.
The pipeline is written using Groovy syntax, usually in a **Jenkinsfile**.
It allows the entire workflow to be version-controlled and repeatable.
Jenkins Pipeline makes software delivery faster, reliable, and consistent.

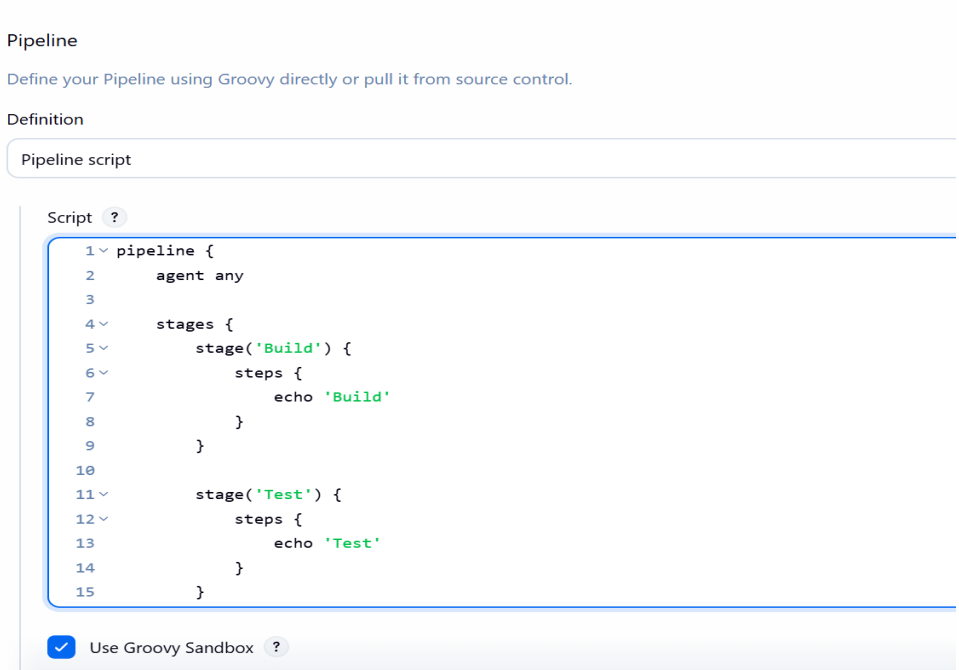
2. Scripted vs Declarative Pipeline

Scripted Pipeline	Declarative Pipeline
Uses Groovy-based scripting with full programming control.	Uses a **simple, structured syntax** with predefined rules.
Very flexible and powerful for complex logic.	**Less flexible** but easier to write and understand.
Syntax is written inside node {} blocks.	Syntax starts with the pipeline {} block.
Best suited for experienced Jenkins users.	Best suited for **beginners and standard CI/CD tasks**.
Errors are harder to detect due to free-form scripting.	Built-in validation makes **error detection easier**.

3. Create **Pipeline Job**



4. Write pipeline in Jenkins UI



5. Add stages:
    - Build
    - Test
    - Deploy

Sample Task

```
pipeline {
  agent any
  stages {
    stage('Build') { steps { echo 'Build' } }
    stage('Test')  { steps { echo 'Test' } }
    stage('Deploy'){ steps { echo 'Deploy' } }
  }
}
```

Each **stage** represents one step in the CI/CD process.

- **Build** → Compiles or builds the application
- **Test** → Runs automated tests
- **Deploy** → Deploys the application

6. Introduce:
    - `agent`
    - `stages`
    - `steps`

agent

- Defines **where** the pipeline runs
- agent any → run on any available Jenkins node

stages

- Groups all pipeline stages

steps

- Contains actual commands or actions

```
Started by user Sivashree
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in C:\ProgramData\Jenkins\.jenkins\workspace\Pipeline-demo
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Build)
[Pipeline] echo
Build
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Test)
[Pipeline] echo
Test
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Deploy)
[Pipeline] echo
Deploy
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Sample Task

```
pipeline {
  agent any
  stages {
    stage('Build') { steps { echo 'Build' } }
    stage('Test')  { steps { echo 'Test' } }
    stage('Deploy'){ steps { echo 'Deploy' } }
  }
}
```

Outcome

Learner writes basic pipelines confidently.

5. Learning Goal

Pipeline as code from repository

Stepwise Tasks

1. Create `Jenkinsfile`

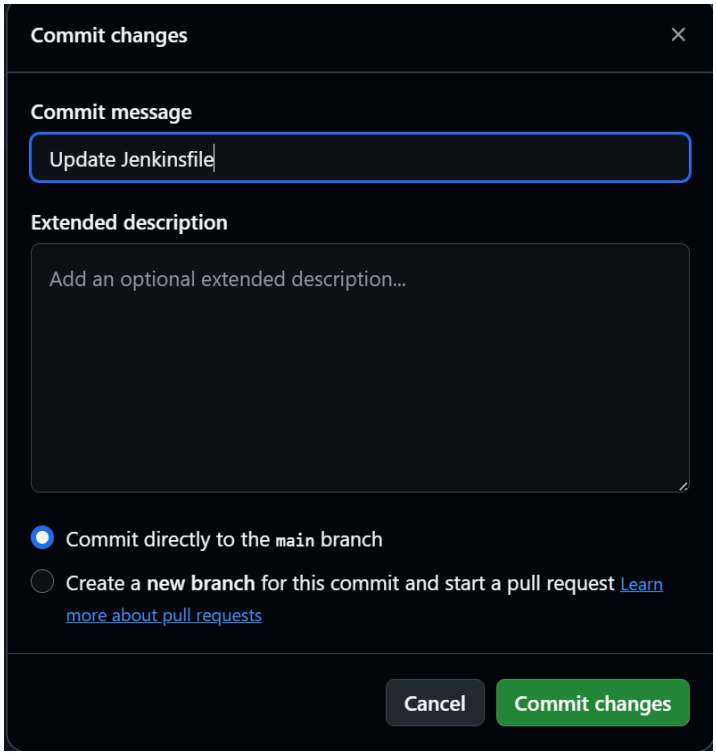
main git_jenkins / Jenkinsfile

Sivashree-117 Create Jenkinsfile for CI/CD pipeline

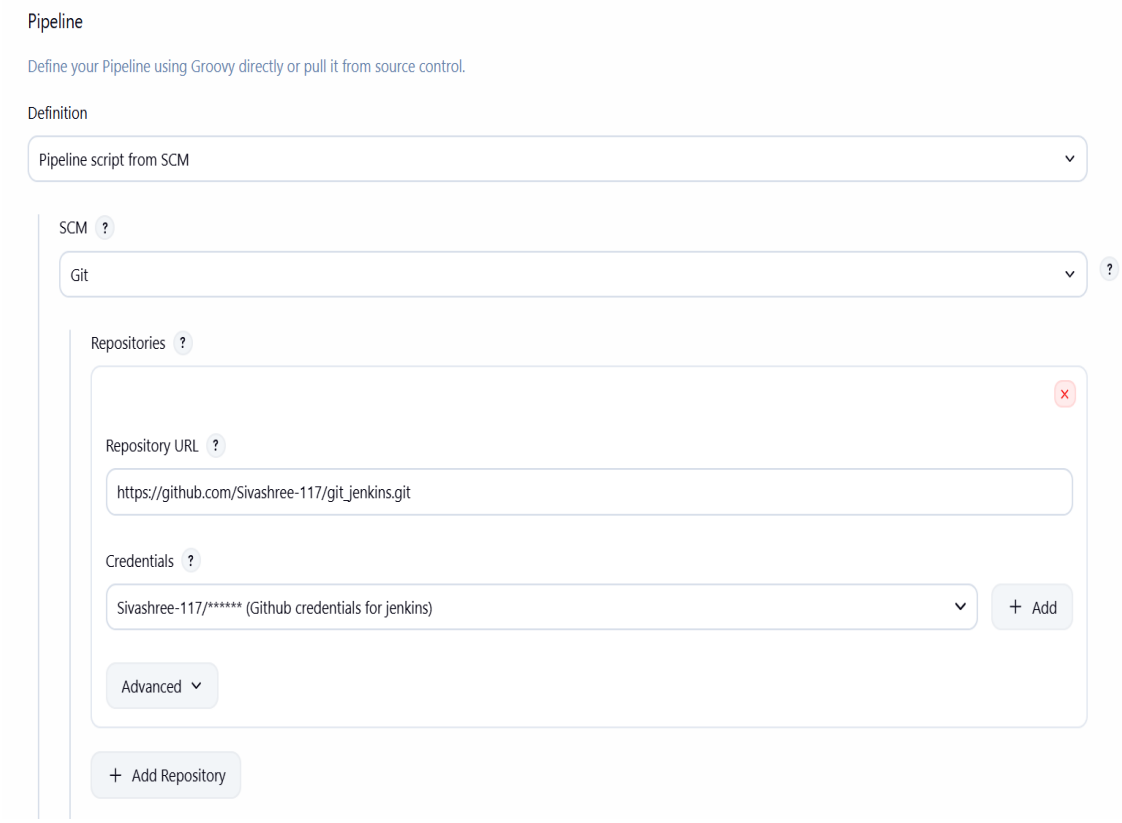
Code Blame 20 lines (20 loc) · 385 Bytes

```
pipeline {
    agent any
    stages {
        stage('Build') {
            steps {
                echo 'Building application'
            }
        }
        stage('Test') {
            steps {
                echo 'Testing application'
            }
        }
        stage('Deploy') {
            steps {
                echo 'Deploying application'
            }
        }
    }
}
```

2. Commit Jenkinsfile to GitHub



3. Configure Pipeline job:
   - SCM → Git



4. Trigger build automatically

Triggers

Set up automated actions that start your build based on specific events, like code changes or scheduled times.

- [ ] Build after other projects are built ?
- [ ] Build periodically ?
- [x] GitHub hook trigger for GITScm polling ?
- [ ] Poll SCM ?
- [ ] Trigger builds remotely (e.g., from scripts) ?

5. Modify Jenkinsfile and rebuild

git_jenkins / Jenkinsfile in main

Edit Preview

```
pipeline {
    agent any
    stages {
        stage('Build') {
            steps {
                echo 'Building application'
            }
        }
        stage('Test') {
            steps {
                echo 'Testing application'
            }
        }
        stage('Deploy') {
            steps {
                echo 'Deploying application'
            }
        }
    }
}
echo 'Build process completed successfully'
echo '23MIS0117 Sivashree'
```

Commit changes

Commit message

Add echo statement for user identification

Extended description

```
Building application
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Test)
[Pipeline] echo
Testing application
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Deploy)
[Pipeline] echo
Deploying application
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] echo
Build process completed successfully
[Pipeline] echo
23MIS0117 Sivashree
[Pipeline] End of Pipeline
Finished: SUCCESS
```

6. View Stage View

Pipeline-From-Git

Stage View

	Declarative: Checkout SCM	Build	Test	Deploy
Average stage times: (full run time: ~7s)	2s	165ms	126ms	131ms
#2 Feb 04 22:35 — 2 commits	2s	165ms	124ms	124ms
#1 Feb 04 22:30 — No Changes	2s	166ms	129ms	138ms

Permalinks

- Last build (#2), 2 min 6 sec ago
- Last stable build (#2), 2 min 6 sec ago
- Last successful build (#2), 2 min 6 sec ago
- Last completed build (#2), 2 min 6 sec ago

Outcome

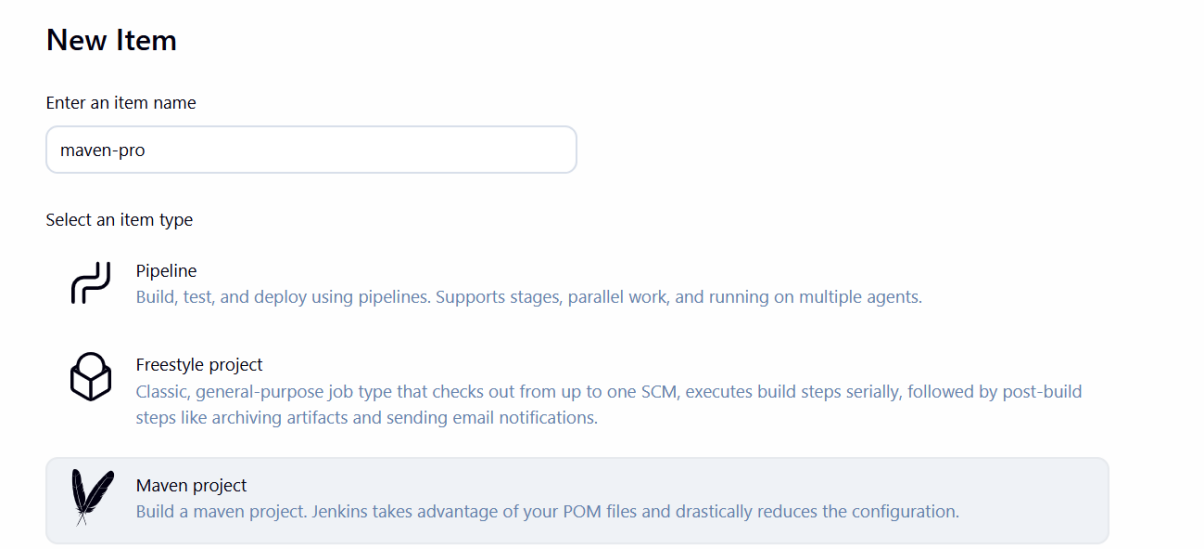
Learner practices **Infrastructure as Code** concept.

6. Learning Goal

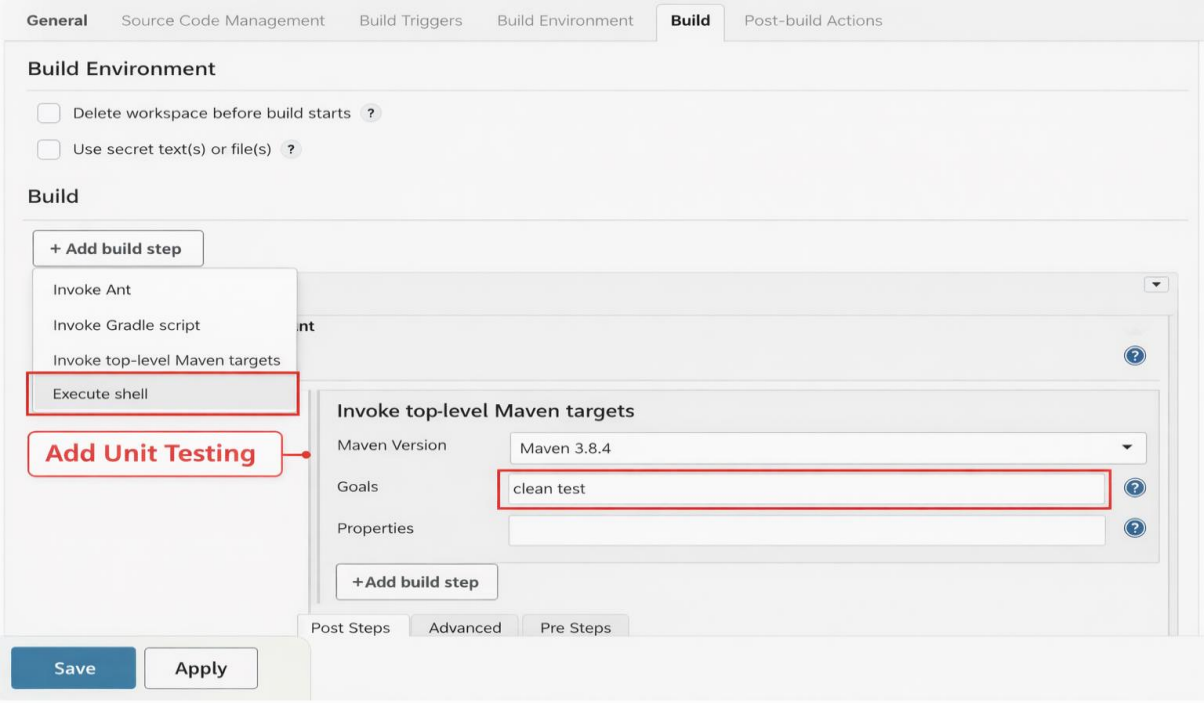
Build **real CI/CD pipelines**

Stepwise Tasks

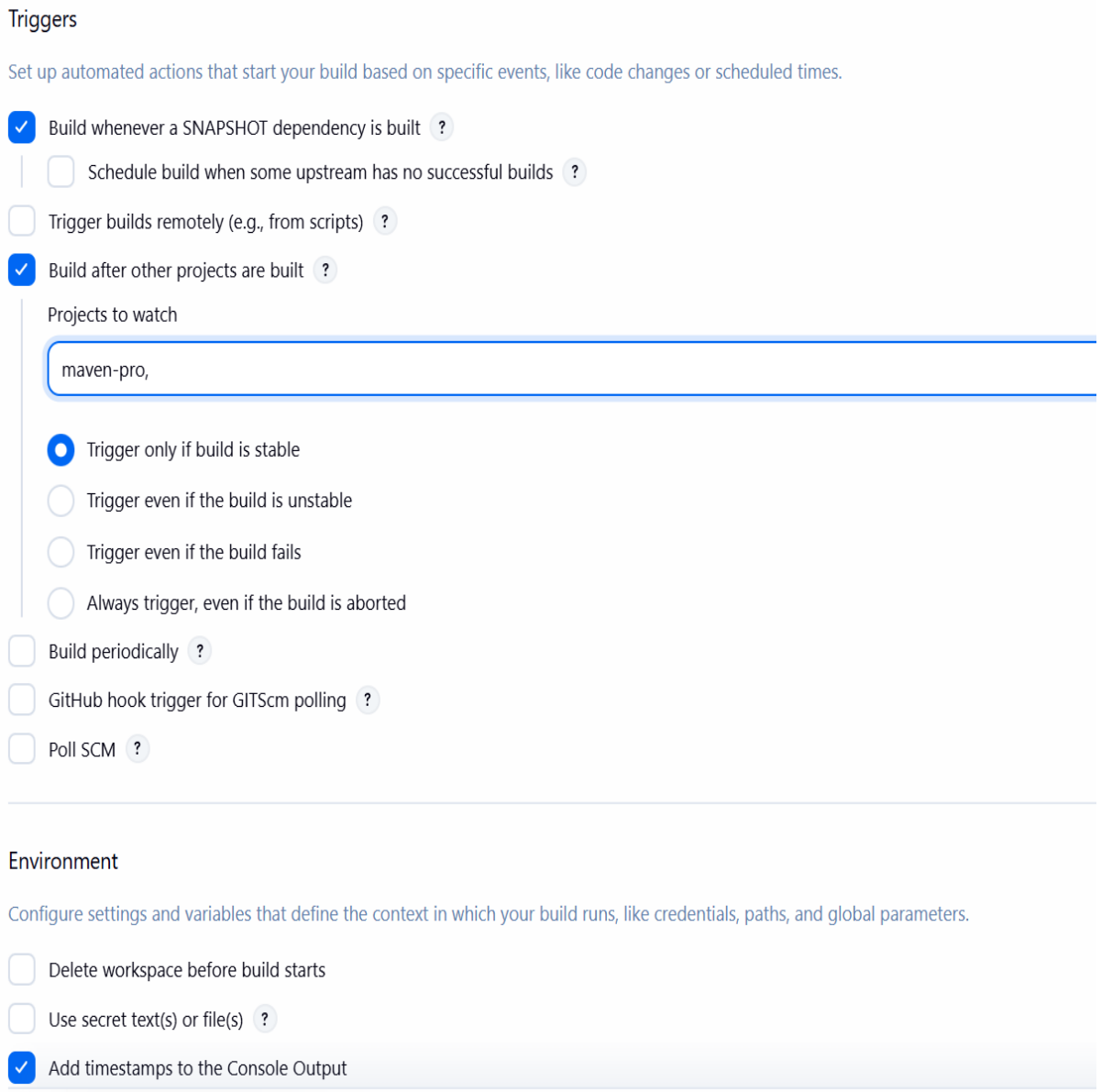
1. Integrate build tools:
   - Maven / Gradle / Node



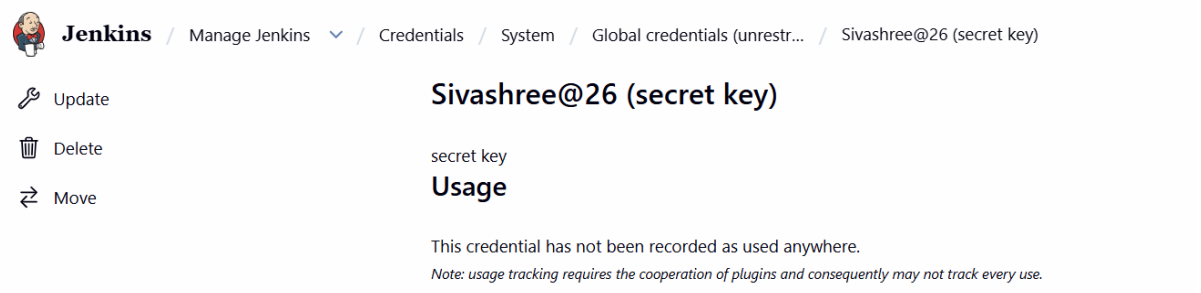
2. Add:
   - Unit tests
   - Code quality (SonarQube)

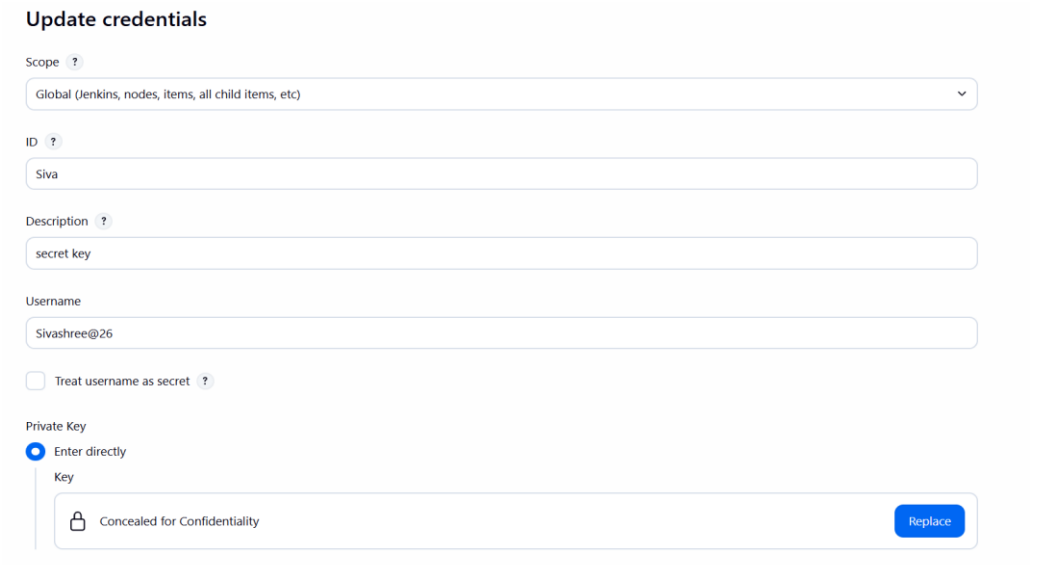


3. Use environment variables

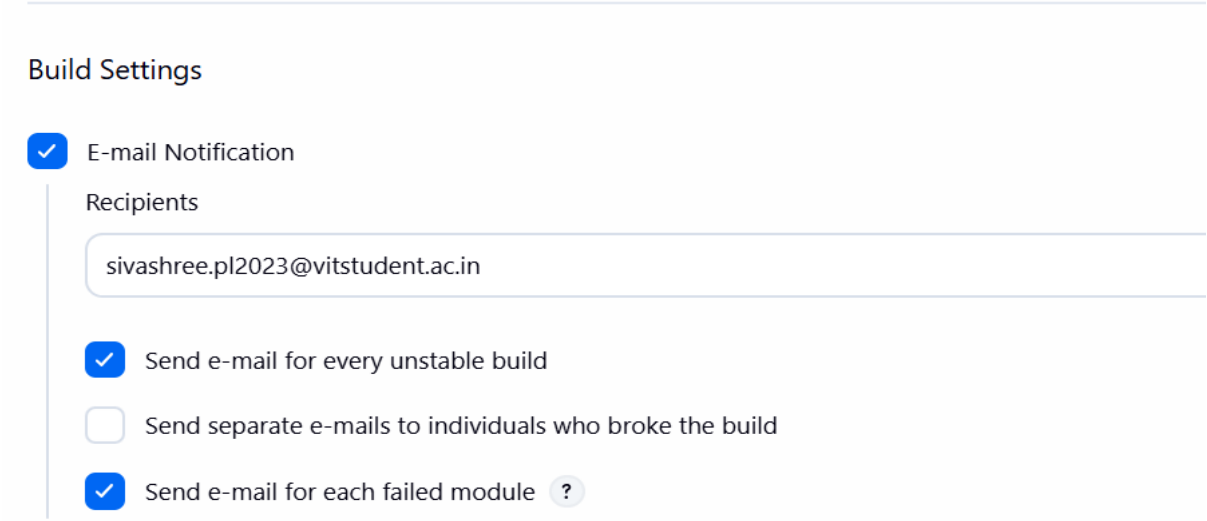


4. Credentials management





5. Notifications (Email / Slack)



Outcome

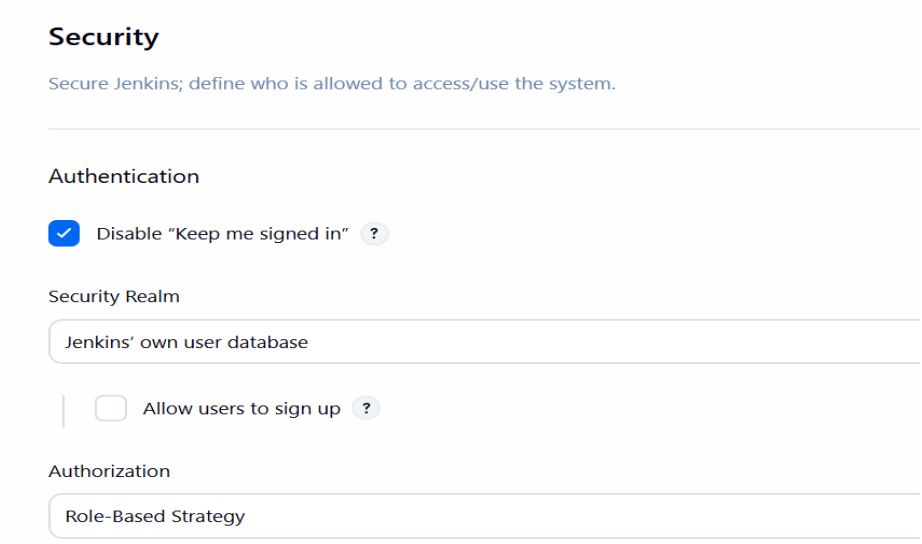
Learner builds production-grade pipelines.

7. Learning Goal

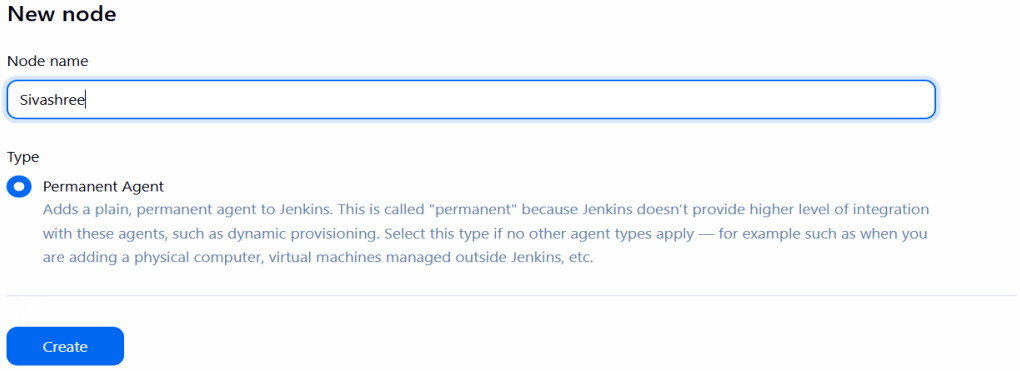
Manage Jenkins infrastructure

Stepwise Tasks

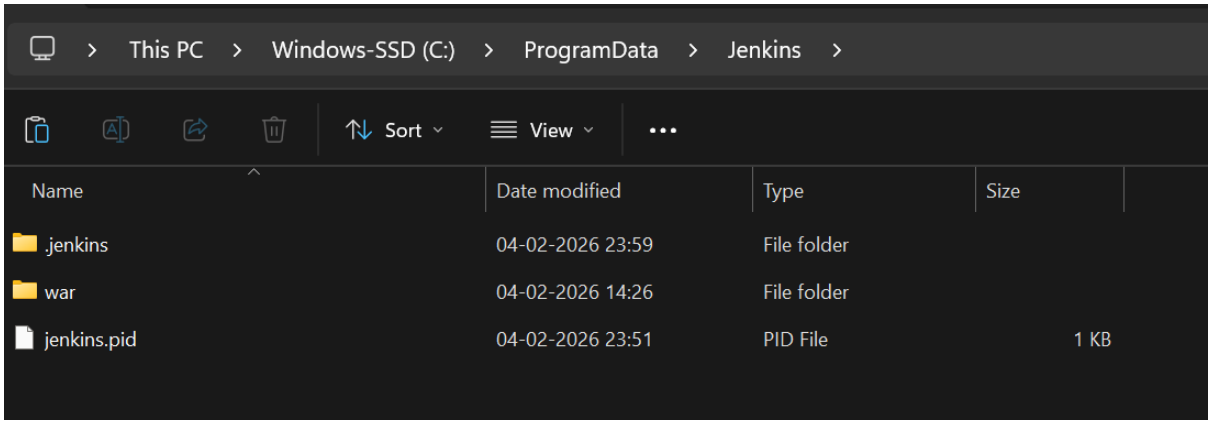
1. User & Role management



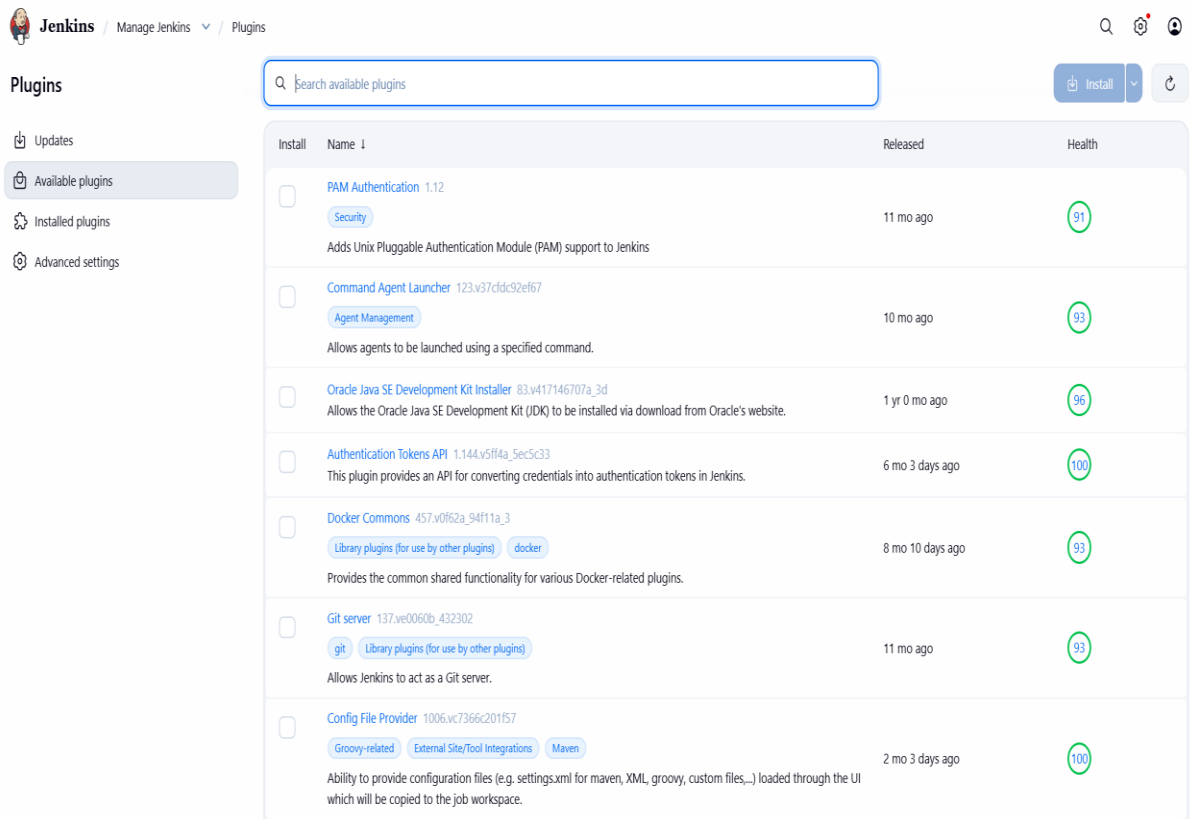
2. Node (Agent) configuration



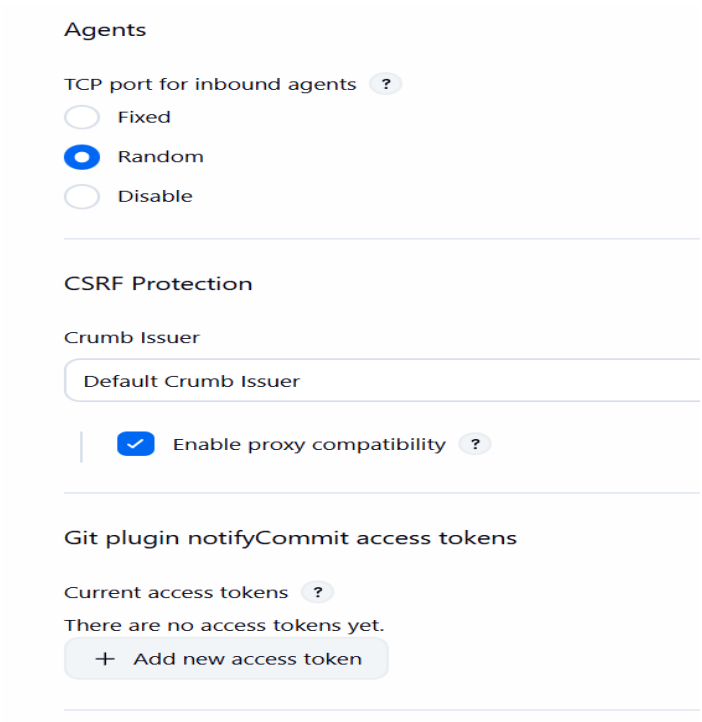
3. Backup & Restore



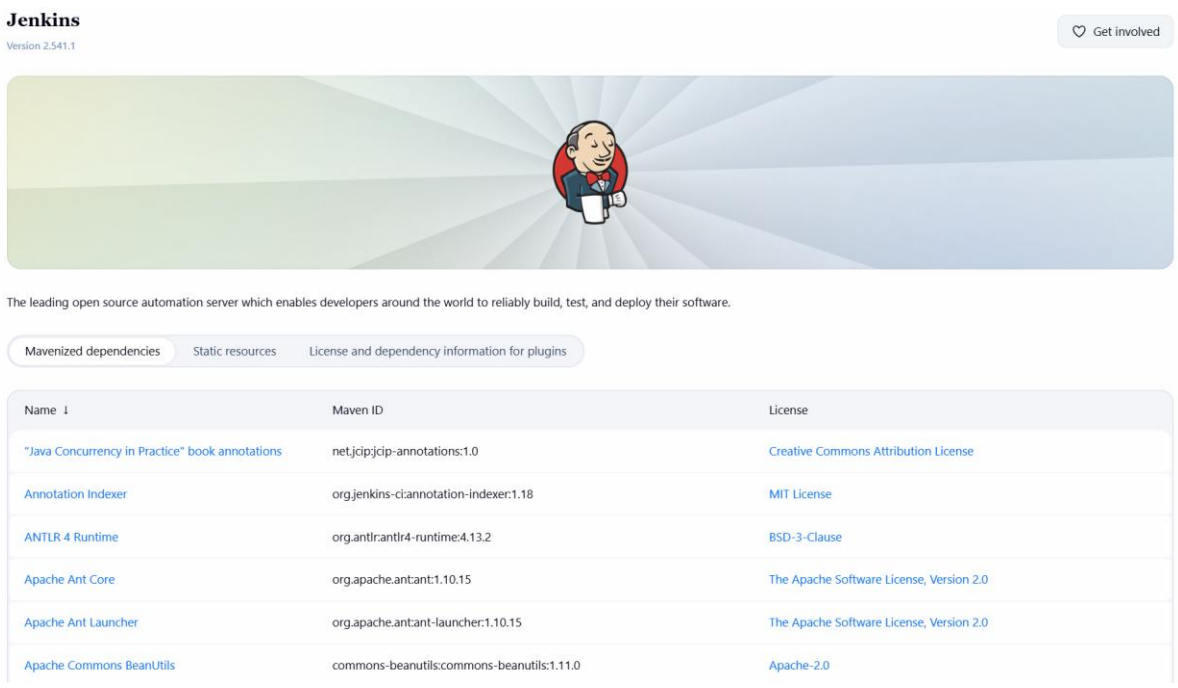
4. Plugin management



5. Security best practices



6. Jenkins upgrade process



✓ Outcome

Learner becomes Jenkins administrator.